

整车CAN开发指南

变更履历

版本号	修订内容说明	作者	审核	批准
V1.0.00	首次发布	刘祖汉	沈向东	沈成宇
V1.1.01	增加VCAN接收和发送常用接口列表	段扬洋	刘祖汉	

目录

1 Canbedded代码生成

- 1.1 Geny工程创建
- 1.2 波特率设置
- 1.3 滤波器设置
- 1.4 动态ID报文句柄设置
- 1.5 IL层接收和发送设置
- 1.6 代码生成

2 应用层代码开发

- 2.1 接收报文制作
 - 2.1.1 常用接收接口列表
 - 2.1.2 接收VCU信号并判断通讯丢失
 - 2.1.3 传递整车上下高压指令
 - 2.1.4 接收TMS信号并判断通讯丢失
 - 2.1.5 其他可选接口
- 2.2 发送报文制作
 - 2.2.1 常用外发接口列表
 - 2.2.2 充电状态发送
 - 2.2.3 高压状态发送
 - 2.2.4 详细电压发送（单帧循环）
 - 2.2.5 详细电压发送（动态 ID）
 - 2.2.6 详细温度发送（单帧循环）
 - 2.2.7 详细温度发送（动态 ID）

3 整车CAN测试

- 3.1 代码静态测试
- 3.2 代码动态测试
- 3.3 HIL测试
- 3.4 实车调试

1 Canbedded代码生成

1.1 Geny工程创建

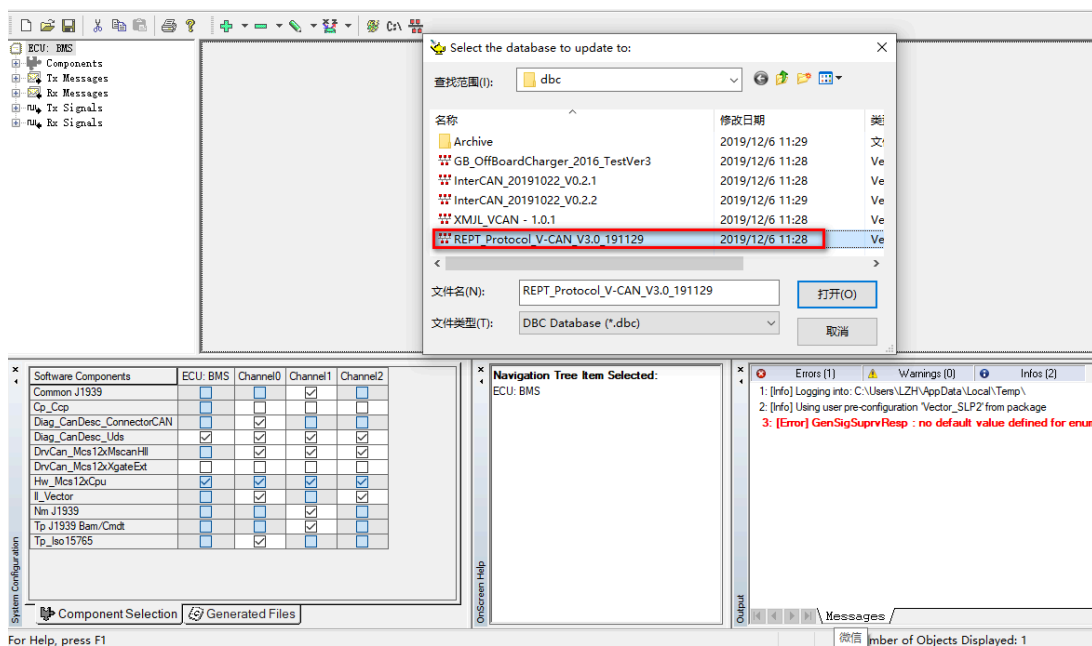
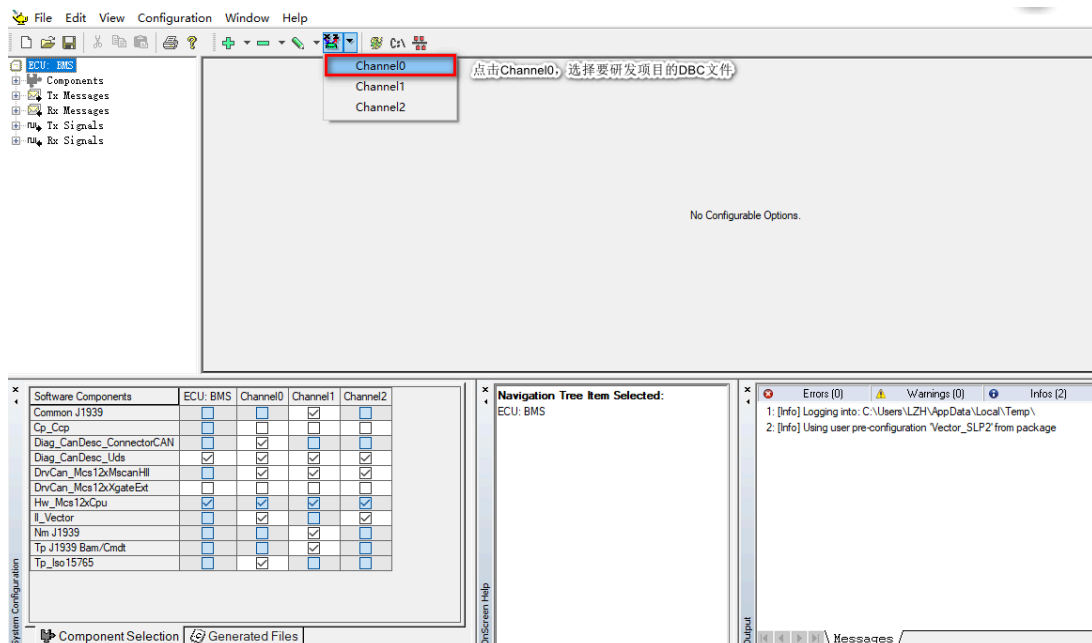
克隆平台代码至项目文件夹，将整车CAN DBC文件放入平台dbc文件夹中，

bin	2019/12/6 12:42	文件夹	
BusPlatform.si4project	2019/12/6 17:03	文件夹	
CANbedded	2019/12/6 11:28	文件夹	
cfg	2019/12/6 11:28	文件夹	
cmd	2019/12/6 11:28	文件夹	
config	2019/12/6 11:28	文件夹	
dbc	2019/12/6 11:31	文件夹	
Platform4.0_Data	2019/12/3 12:31	文件夹	
prm	2019/12/6 11:28	文件夹	
Sources	2019/12/6 11:28	文件夹	
.cproject	2019/3/12 20:53	CPROJECT 文件	6 KB
.gitignore	2019/12/6 11:28	GITIGNORE 文件	1 KB
.project	2019/3/12 20:53	PROJECT 文件	1 KB
C_Layout.hwl	2019/12/6 11:28	HWL 文件	2 KB
Default.mem	2019/12/6 11:28	MEM 文件	1 KB
DevDetails	2019/12/6 11:28	Microsoft Excel ...	9 KB
Full_Chip_Simulation	2019/12/6 11:28	配置设置	2 KB
P&E_Multilink_CyclonePro	2019/12/6 11:28	配置设置	4 KB
Platform4.0	2019/12/6 11:28	CodeWarrior Pr...	199 KB
TBDML.hwc	2019/12/6 11:28	HWC 文件	0 KB
TBDML	2019/12/6 11:28	配置设置	4 KB
软件说明	2019/12/6 11:28	文本文档	1 KB

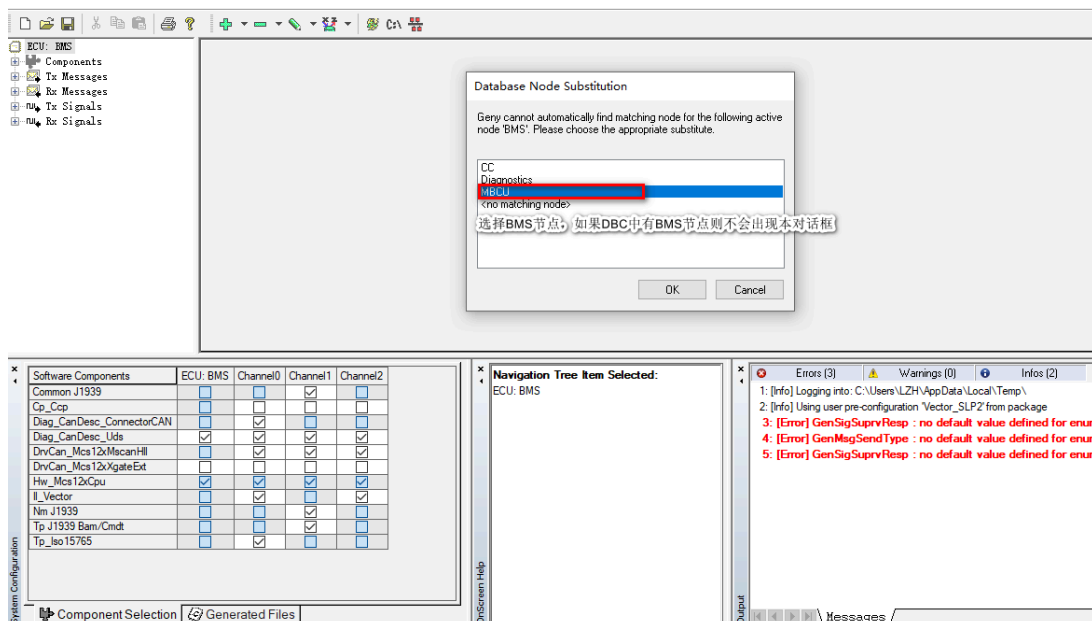
打开平台Geny工程,

Archive	2019/12/6 11:29	文件夹	
GB_OffBoardCharger_2016_TestVer3	2019/12/6 11:28	Vector DBC-File	21 KB
GB_OffBoardCharger_2016_TestVer3	2019/12/6 11:28	配置设置	6 KB
InterCAN_20191022_V0.2.1	2019/12/6 11:28	Vector DBC-File	177 KB
InterCAN_20191022_V0.2.1	2019/12/6 11:29	配置设置	5 KB
InterCAN_20191022_V0.2.2	2019/12/6 11:29	Vector DBC-File	178 KB
InterCAN_20191022_V0.2.2	2019/12/6 11:29	配置设置	5 KB
J1939GenInterface	2019/12/2 19:45	XML 文档	127 KB
Load_migration_7.0.120_to_9.1.105_Pl...	2019/12/6 11:28	文本文档	6 KB
Platform_Template_V4.6_161128	2019/12/6 11:28	CANdela Diagno...	710 KB
Platform_Template_QX 160718	2019/12/6 11:28	CANdela Diagno...	654 KB
XMJL_PRJ	2019/12/6 11:31	GNV 文件	10,977 KB
XMJL_VCAN - 1.0.1	2019/12/6 11:28	Vector DBC-File	29 KB
XMJL_VCAN - 1.0.1	2019/12/6 11:28	配置设置	5 KB

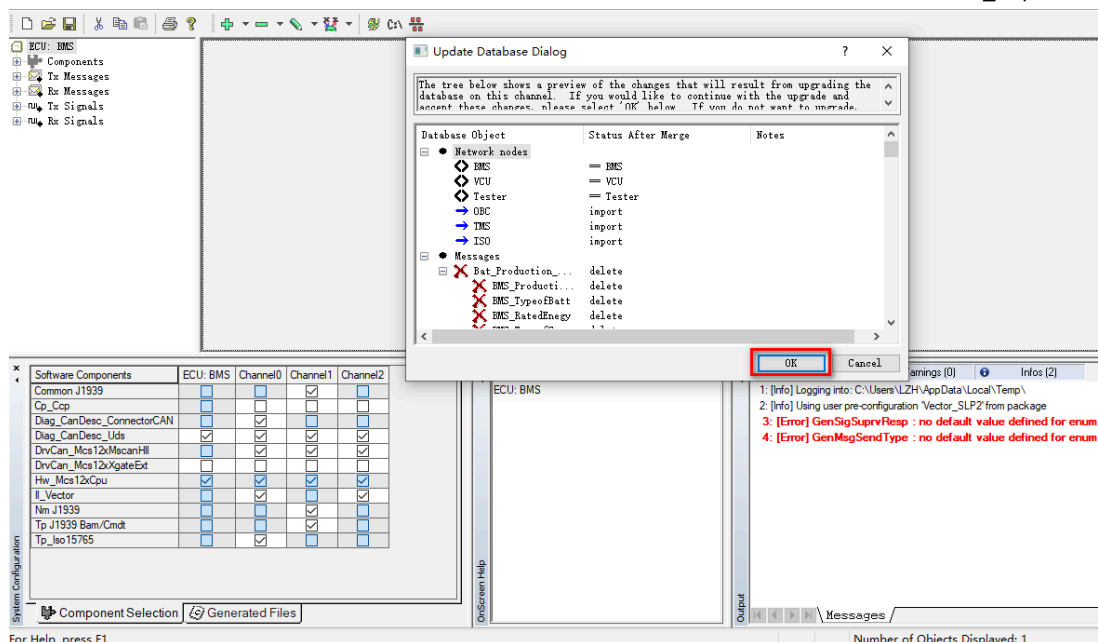
更新Channel0的DBC文件为项目整车CAN DBC,



如果DBC节点中没有BMS节点，Geny会提示选择其中一个节点为BMS，点击OK，



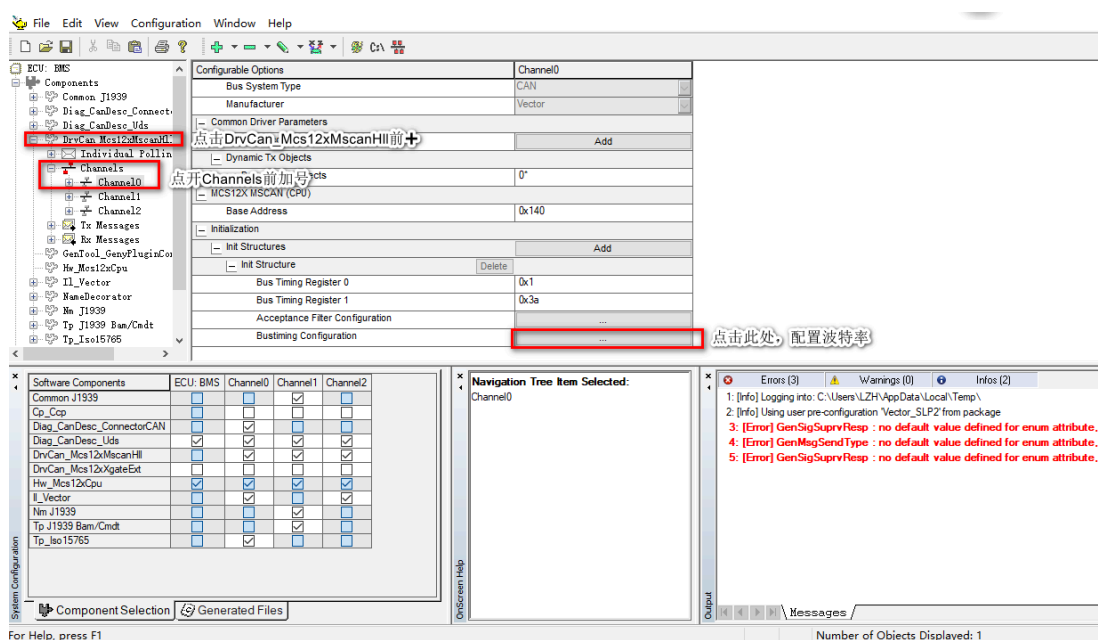
确认报文更改内容，点击OK



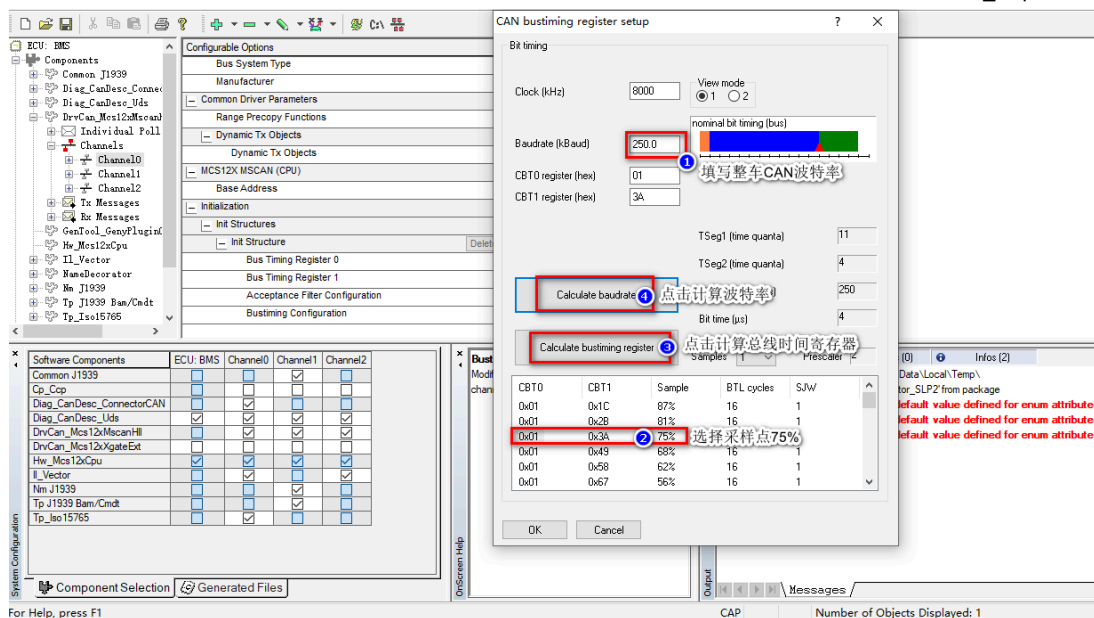
另存Geny工程为项目Geny工程: 文件另存为

1.2 波特率设置

在DrvCan模块选择Channel0配置整车CAN的波特率

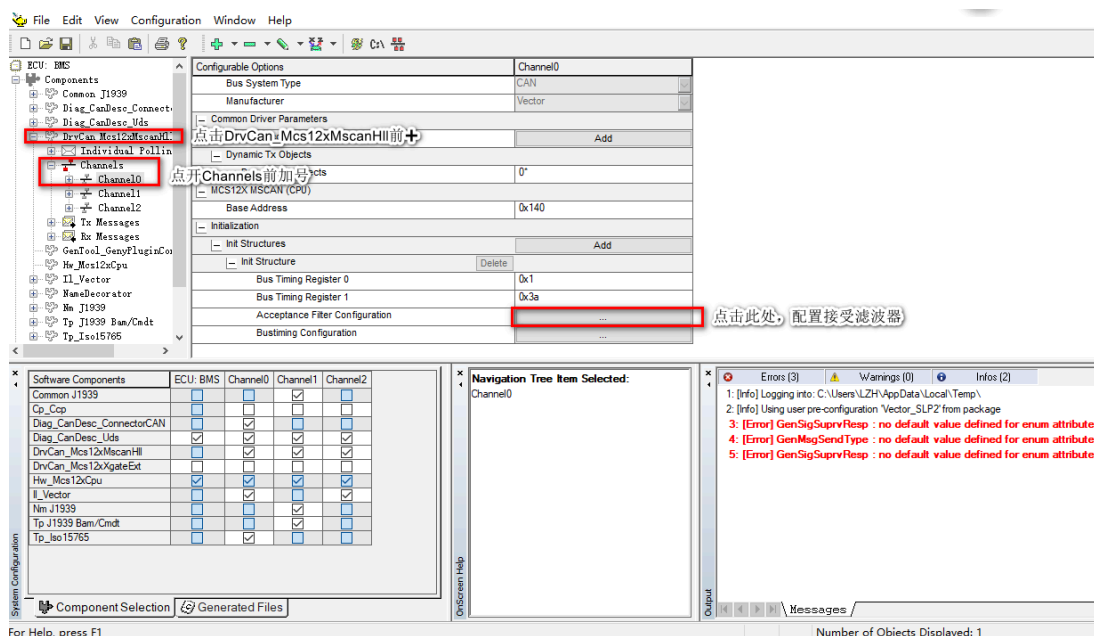


按照如下步骤设置波特率，最后点击OK，波特率设置完毕

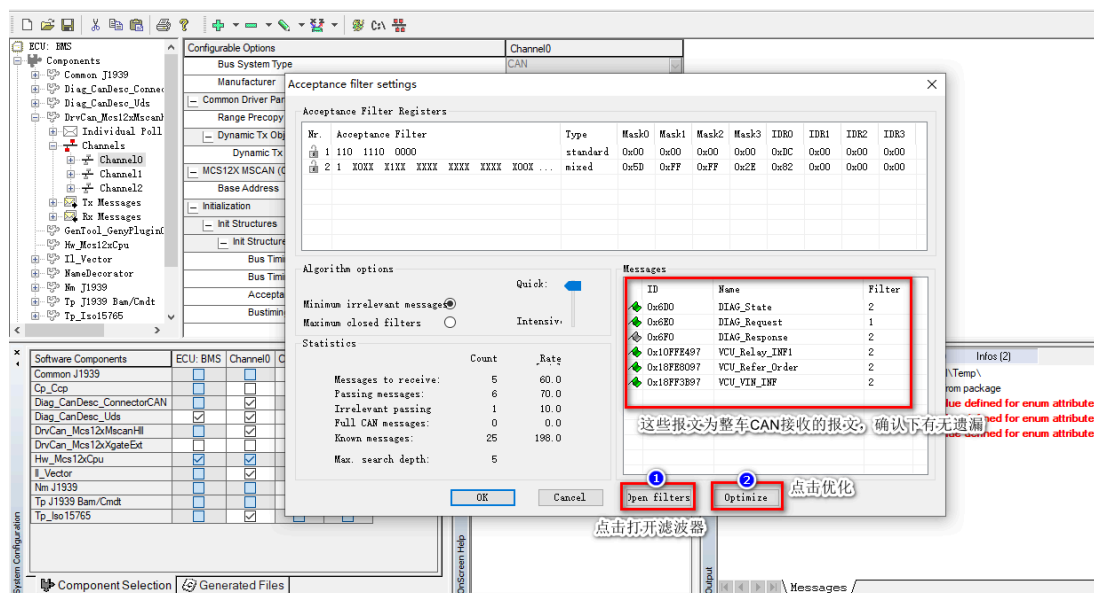


1.3 滤波器设置

在DrvCan模块选择Channel0，配置整车CAN的接受滤波器



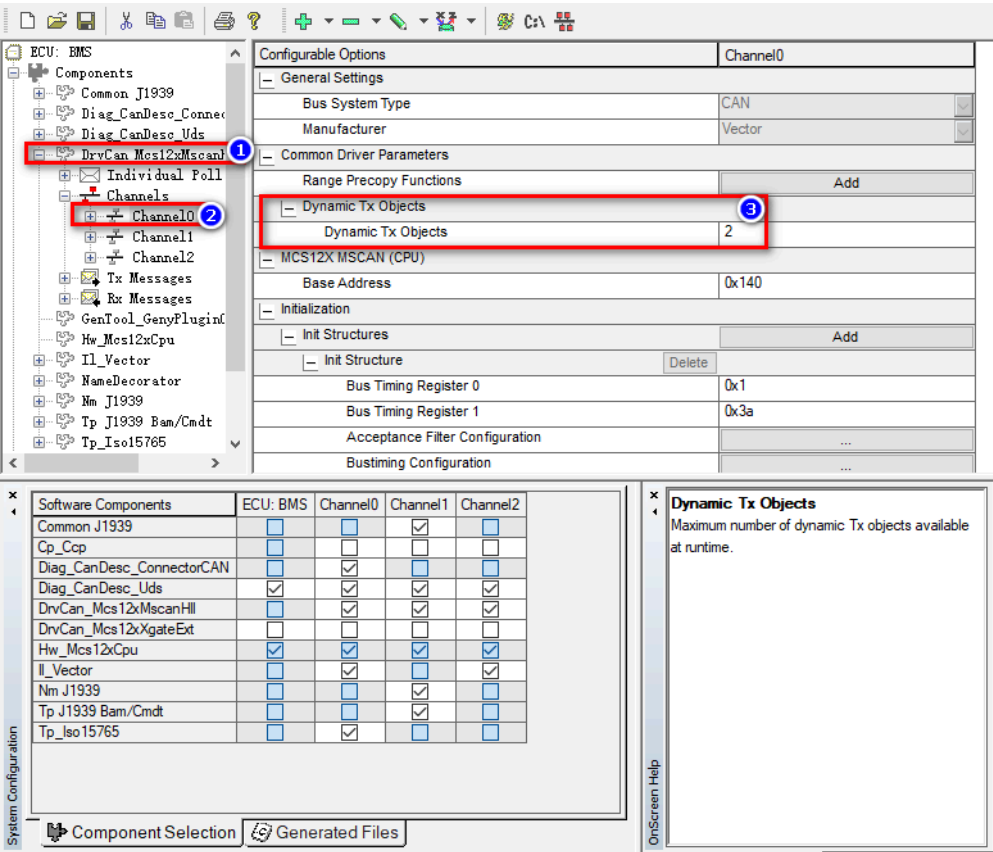
按照如下步骤设置接收滤波器，点击OK，完成配置。



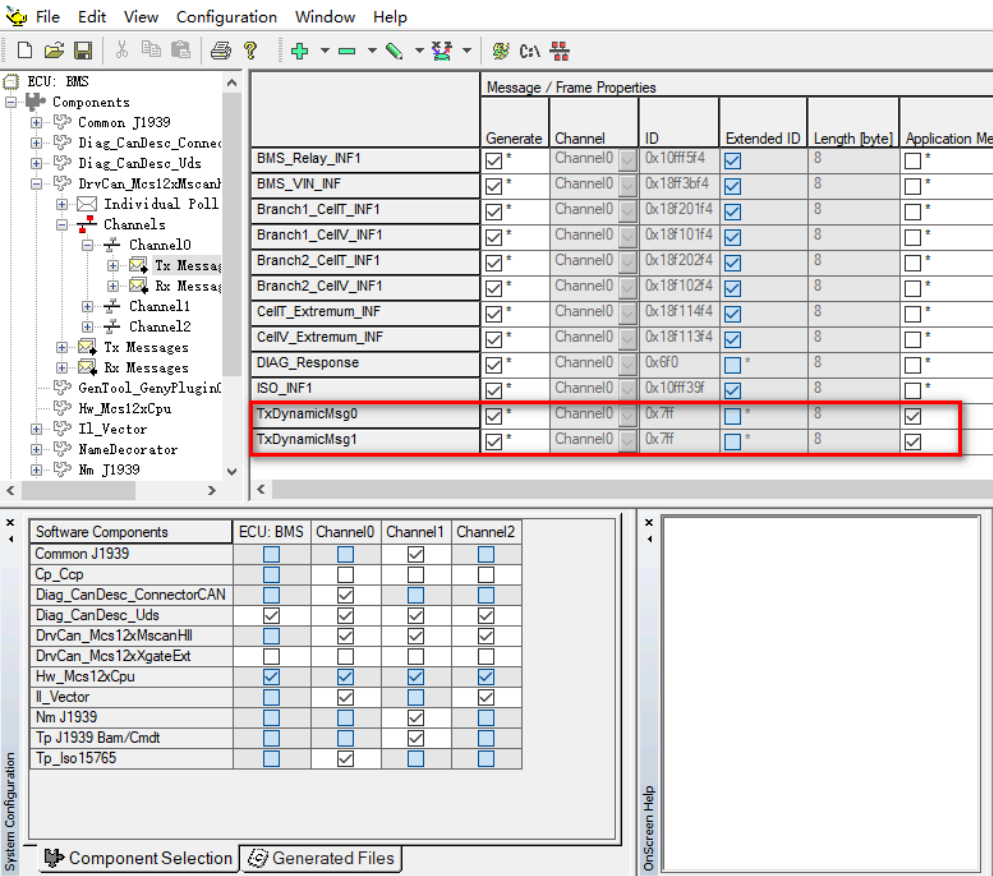
1.4 动态ID报文句柄设置

如果需要动态ID报文，请按照下图配置动态ID句柄数量。

注，现在新版平台需设置动态ID句柄数量大于1，有动态ID的需设置句柄数量大于等于3；

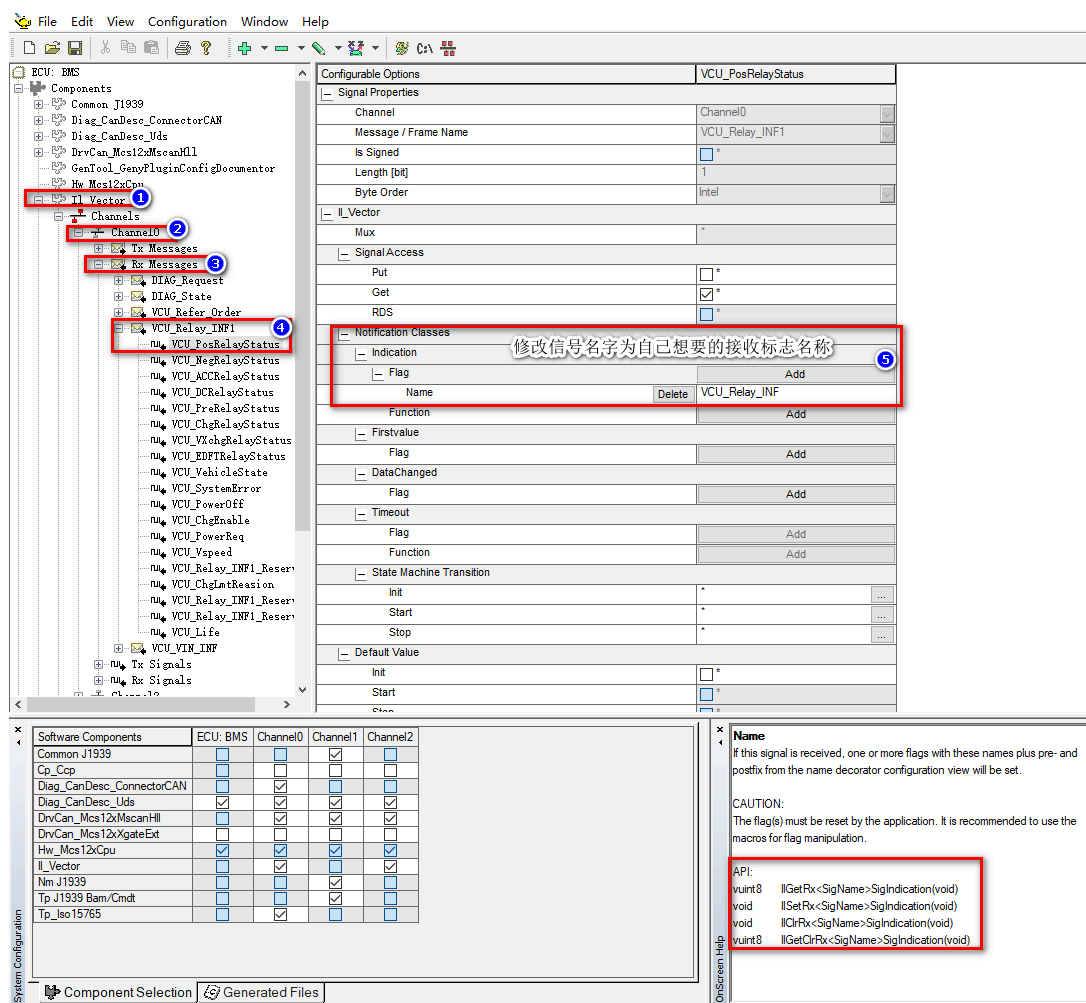


配置完成后，在channel0 Tx Message可以看到添加的句柄如下图所示。



1.5 IL层接收和发送设置

接收标志设置：IL_Vectorchannel0Rx Messages报文名称信号，在Notification Classes 点击Indication\Flag\Name 中Add指令，添加信号接收完成标志。



Canbedded提供如下API由应用层调用，

vuint8 IIGetRx<SigName>SigIndication(void)：底层接收到报文后，该函数返回1

void IIClrRx<SigName>SigIndication(void)：清除接收标志，一般使用情况如下

static void VCanVCU2BMSReceive(void)

```
{
    static INT16U VCU_timeout_timer;

    if(IIGetVCU1Indication()) /*接收标志调用*/
    {
        VCAN_Vhc_MainCnST = IIGetRxVCU1_VehicleCntST();
        VCAN_VCU_HVCmd = IIGetRxVCU1_HVOnCmd();
        IIClrVCU1Indication(); /*清除接收标志*/
        (void)Stb_MS_SetTimer(&VCU_timeout_timer);
        VCAN_VCUCOMLost_Flag = FALSE;
    }
    else
    {
        if(RteGetACChgCnntST())
        {
            VCAN_VCUCOMLost_Flag = FALSE;
        }
    }
}
```

```

(void)Stb_MS_SetTimer(&VCU_timeout_timer);

}

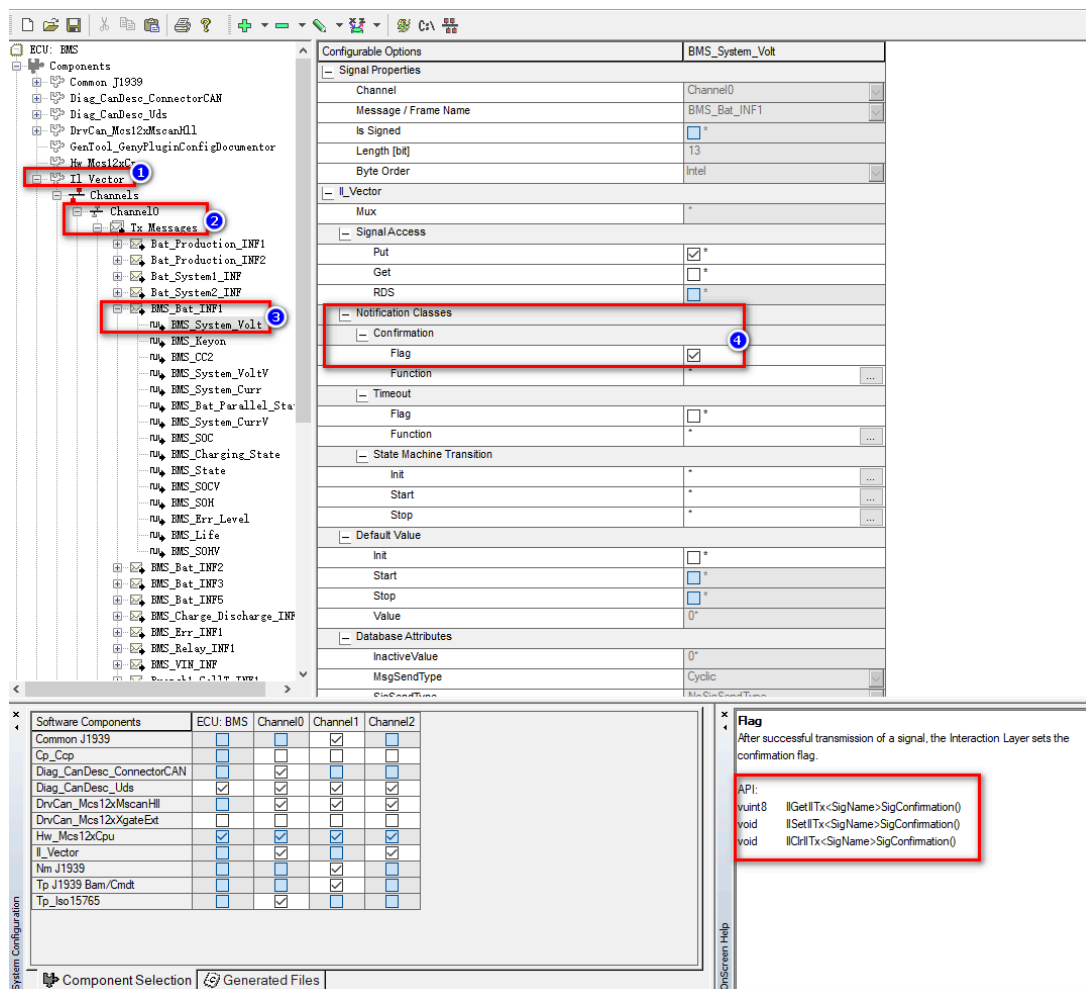
else if(FALSE == VCAN_VCUCOMLost_Flag) /*报文超时判断*/
{
    if(Stb_MS_Timeout(VCU_timeout_timer, VCU_TIMEOUT))
    {
        VCAN_VCUCOMLost_Flag = TRUE;
    }
}

else
{
    (void)Stb_MS_SetTimer(&VCU_timeout_timer);
}
}

ApiPutVcuCommErr(VCAN_VCUCOMLost_Flag);
}

```

发送标志设置：IL_Vectorchannel0Tx Messages报文名称信号，在Notification Classes 点击Confirmation\Flag中的复选框。



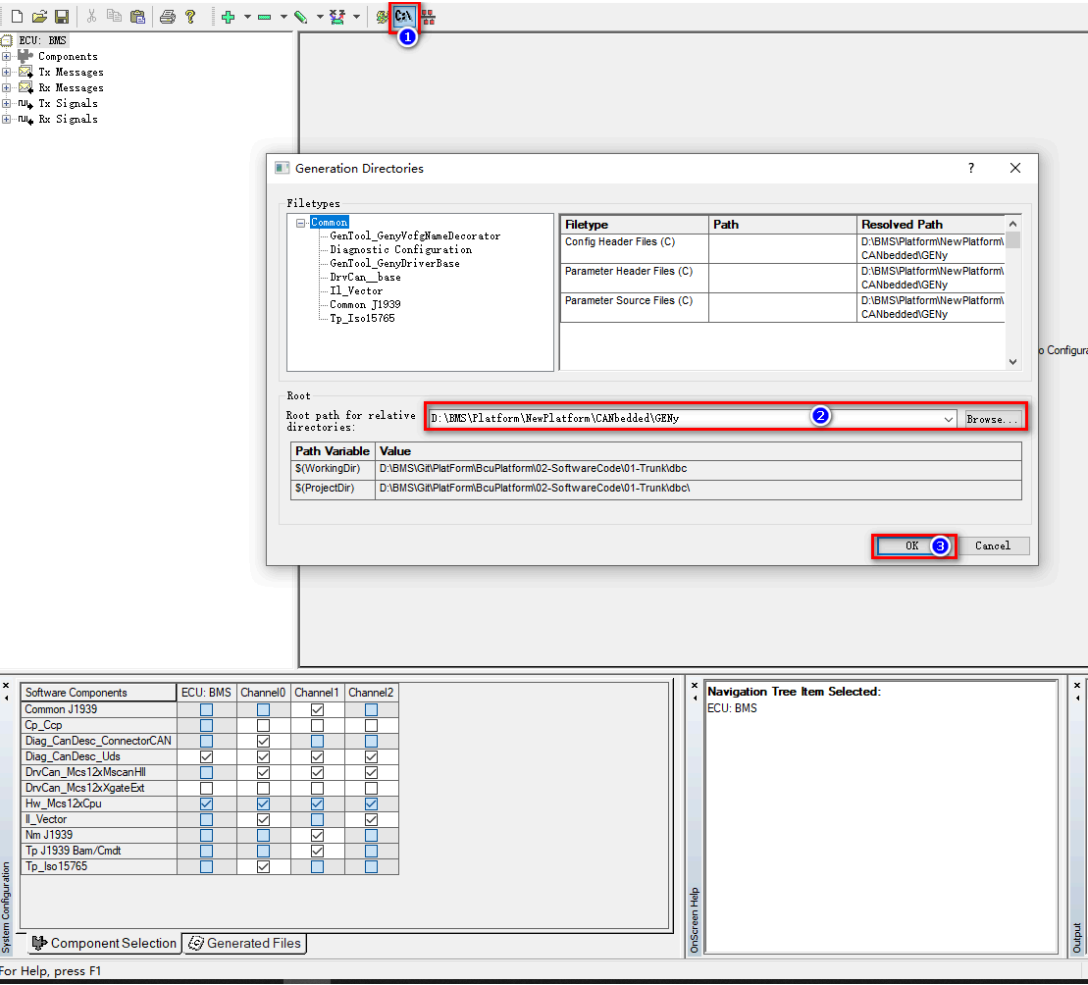
Canbedded提供如下API由应用层调用，

uint8 ILGetITx<SigName>SigConfirmation(): 报文发送完成后，该函数返回1

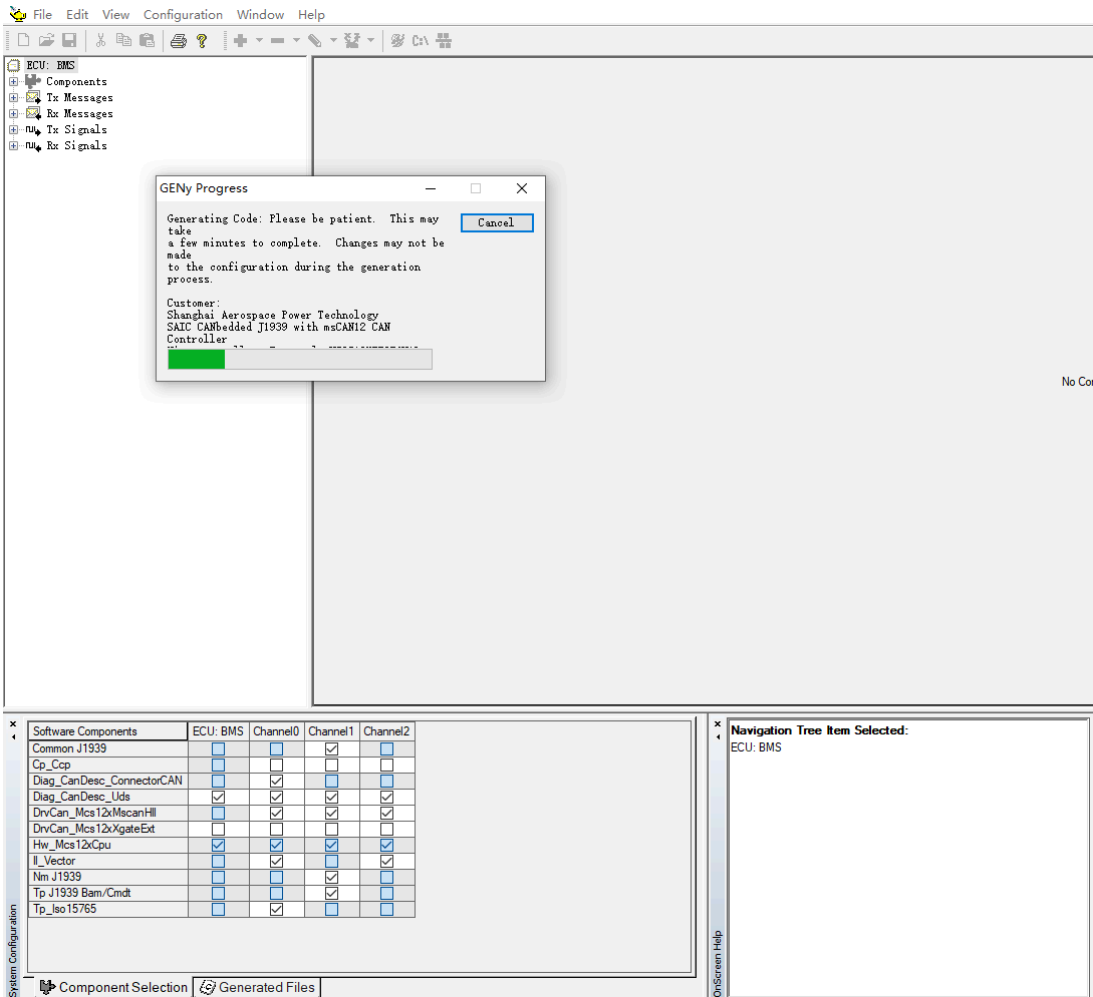
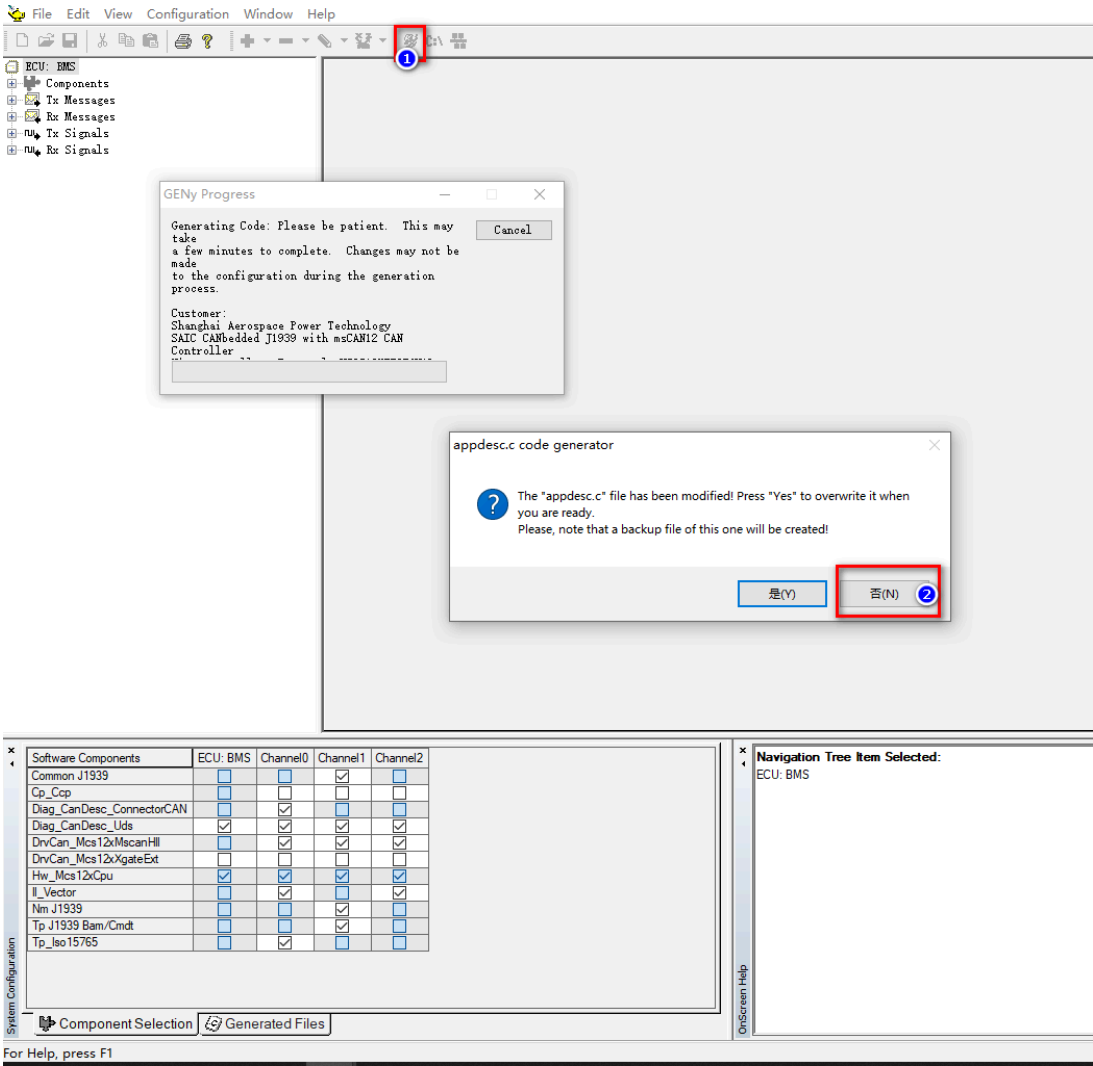
void ILClrITx<SigName>SigConfirmation(): 清除发送完成标志，使用情况类似接收完成标志

1.6 代码生成

代码生成路径设置：点击路径设置，选择项目工程所在文件夹，设置文件夹最终路径为Canbedded\GENy,点击OK，确认文件夹路径。

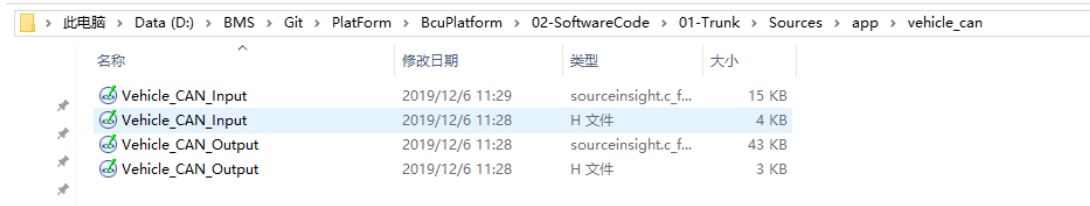


点击生成代码，弹出对话框是否覆盖“appdesc.c”，选择否。等待代码生成完成即可。



2 应用层代码开发

整车CAN模块四个文件，Vehicle_CAN_Input.c制作BMS从整车CAN接收的报文，Vehicle_CAN_Output.c制作BMS发送至整车CAN的报文。



2.1 接收报文制作

2.1.1 常用接收接口列表

接口数据类型/原型	说明
void ApiPutHVCmd(INT8U cmd)	传递整车上下高压指令（建议：整车侧指令先做Debounce/状态机处理后再调用该接口）
INT8U ApiPutMainNegCheckDisable(INT8U DisableCmd)	禁止主负继电器做粘连检测的命令（用于BMS检整车主正继电器拓扑等场景）
void ApiPutPowerOffCmd(INT8U cmd, VCU_CMD src)	传递整车准备切断低压供电命令；整车发出后约5s切断低压（VCU_CMD为宏：来自VCU的低压下电指令源）
void ApiPutHVEmergOffCmd(BOOLEAN EmergOffCmd)	传递整车紧急下高压指令；收到后直接断开继电器，不做电流判断（谨慎使用）
void ApiPutVcuCommErr(INT8U data)	VCU通讯丢失故障标志（建议丢失判定阈值：5~10倍报文周期）；data: 0=正常, 1=丢失
void ApiPutThml_VehicleSpeed(INT16U VehicleSpeed)	整车车速信息；单位km/h；分辨率1
void ApiPutKeyonLostErr(INT8U data)	KeyOn丢失故障标志（充放电过程中需结合整车是否持续发上高压指令进行判断）；data: 0=正常, 1=丢失
void ApiPutVinByte(INT8U index, INT8U VinByte)	接收整车VIN码字节；建议收齐后与RteGetVinByte(i)对比，不一致再更新，避免重复刷写
void ApiPutVCUDCChgActive(INT8U cmd)	允许闭合直流充电继电器指令；BRO阶段BMS收到后才闭合直流充电继电器
void ApiPutVCUDCChgAllow(INT8U cmd)	允许进入直流充电指令；BRO阶段收到后BMS才发送0xAA进入充电流程；充电中若不允则退出直流充电并上报故障
void ApiPutTMSCommErr(INT8U data)	TMS通讯丢失故障标志（有TMS时）；data: 0=正常, 1=丢失；可结合TMS唤醒状态决定是否判定
void ApiPutThml_TMSPower(INT16U Tmspower)	TMS需求功率；单位kW；分辨率0.1
void ApiPutThml_TMSHVS(INT8U RelaySt)	TMS高压状态；0=断开, 1=闭合
void ApiPutThml_TMSFaultLvl(INT8U TmsfltLvl)	TMS故障等级；1=轻微；2=中等（TMS高压断开）；3=严重（TMS下电）
void ApiPutThml_TMSInletTemp(INT8U InletTemp)	TMS进水口温度；单位℃；分辨率1；偏移-40℃
void ApiPutThml_TMSOutletTemp(INT8U OutletTemp)	TMS出水口温度；单位℃；分辨率1；偏移-40℃
void ApiPutOBCCommErr(INT8U data)	OBC通讯丢失故障标志（有OBC时）；非充电状态建议不做丢失判断；data: 0=正常, 1=丢失
void ApiPutOBCFaultLvl(INT8U faultlvl)	OBC故障等级；1=轻微；2=严重（停止充电）
void ApiPutACChgCnntST(INT8U conntst)	交流充电枪连接状态（OBC检测）；0=未连接, 1=已连接
void ApiPutVCUACChgAllow(INT8U cmd)	整车允许进入交流充电指令（可选）

void ApiPutInsulationCommErr(INT8U data)	绝缘单元通讯丢失故障标志（绝缘仪在整车CAN网络时）； data: 0=正常, 1=丢失
void ApiPutExtSystemIsoValue(INT16U isovalue)	外部绝缘仪检测绝缘值；单位kΩ；分辨率1

2.1.2 接收VCU信号并判断通讯丢失

参考代码示例:

```
``c
static void VCanVCU_Relay_INFReceive(void)
{
    static INT16U VCU_timeout_timer = 0u;

    if (IIGetVCU_Relay_INFIndication())
    {
        /* 正常收到报文：读取信号并清除指示位 */
        VCAN_VCU_HVCmd = IIGetRxVCU_NegRelayStatus(); /* 0: Open; 1: Close */
        IIClrVCU_Relay_INFIndication();

        (void)Stb_MS_SetTimer(&VCU_timeout_timer);
        VCAN_VCUCOMLost_Flag = FALSE;
    }
    else
    {
        /* 未收到报文：在未丢失状态下，超时报文丢失 */
        if (FALSE == VCAN_VCUCOMLost_Flag)
        {
            /* 丢失时间建议 5~10 倍以上报文周期 */
            if (Stb_MS_Timeout(VCU_timeout_timer, VCU_TIMEOUT))
            {
                VCAN_VCUCOMLost_Flag = TRUE;
            }
        }
        else
        {
            (void)Stb_MS_SetTimer(&VCU_timeout_timer);
        }
    }

    ApiPutVcuCommErr(VCAN_VCUCOMLost_Flag);
}
```

```
}
...
```

2.1.3 传递整车上下高压指令

- 不建议把“整车指令”直接传给 `ApiPutHVCmd()`。
- 建议按系统流程做 Debounce/状态机；并在充放电场景下增加 KeyOn 丢失判断逻辑。

参考代码示例:

```
```c
/*
 * 功能：将整车上/下高压指令做debounce与状态机处理。
 * 建议：
 * - 上高压debounce：3~5倍报文周期
 * - 下高压debounce：2~3倍报文周期
 * - KeyOn 丢失后下高压等待：30s（示例）
 */
static void VCanHVCmd(void)
{
 static INT16U hvcmd_timer;
 static BOOLEAN keylock_flag = 0u;
 static INT16U keylost_timer;

 switch (VCAN_HVCMD)
 {
 case HVCMD_OFF:
 {
 if (1u == VCAN_VCU_HVCmd)
 {
 /* Debounce 时间选 3~5 倍报文周期 */
 if (Stb_MS_Timeout(hvcmd_timer, HVON_DEBOUNCE))
 {
 VCAN_HVCMD = HVCMD_ON;
 keylock_flag = 1u;
 (void)Stb_MS_SetTimer(&hvcmd_timer);
 }
 }
 else
 {

```

```
 }
}
else
{
 (void)Stb_MS_SetTimer(&hvcmd_timer);
}
}
break;

case HVCMD_ON:
{
 if (0u == VCAN_VCU_HVCmd)
 {
 /* Debounce 时间选 2~3 倍报文周期 */
 if (Stb_MS_Timeout(hvcmd_timer, HVOFF_DEBOUNCE))
 {
 VCAN_HVCMD = HVCMD_OFF;
 (void)Stb_MS_SetTimer(&hvcmd_timer);
 }
 }
 else if (TRUE == VCAN_KeyonLost_Flag)
 {
 /* KeyOn 丢失时，等待 30s 再下高压 */
 if (Stb_S_Timeout(keylost_timer, KEYON_LOST_HVOFFTIME))
 {
 VCAN_HVCMD = HVCMD_OFF;
 }
 }
 else
 {
 (void)Stb_MS_SetTimer(&hvcmd_timer);
 }
}
break;

default:
{
 VCAN_HVCMD = HVCMD_OFF;
```

```

 (void)Stb_MS_SetTimer(&hvcmd_timer);
 }
 break;
}
ApiPutHVCmd(VCAN_HVCMD);

if ((RteGetVmsKeyState() == 0u)
 && (RteGetFcgKeyState() == 0u)
 && (keylock_flag == 1u)
 && (VCAN_HVCMD == HVCMD_ON))
{
 if (Stb_S_Timeout(keylost_timer, KEYON_LOST_CONFTIME))
 {
 VCAN_KeyonLost_Flag = TRUE;
 (void)Stb_S_SetTimer(&keylost_timer);
 }
}
else
{
 (void)Stb_S_SetTimer(&keylost_timer);
 VCAN_KeyonLost_Flag = FALSE;
}

ApiPutKeyonLostErr(VCAN_KeyonLost_Flag);
}

```

## 2.1.4 接收TMS信号并判断通讯丢失

- 有 TMS 时：接收 TMS 信息；同时做通讯丢失判断。
- 若 TMS 未唤醒/未工作：可不判丢失。

参考代码示例:

```

```c
static void VCanTMS_INFReceive(void)
{
    static INT16U TMS_timeout_timer;

    if (IlGetTMS_InfoIndication())
    {

```

```
IIClrTMS_InfoIndication();
```

```
VCAN_TMS_InputTemp = IIGetRxTMS_InletWaterTemp();
```

```
VCAN_TMS_OutputTemp = IIGetRxTMS_OutletWaterTemp();
```

```
VCAN_TMS_RlyST = IIGetRxTMS_RelayST();
```

```
VCAN_TMS_FaultLvl = IIGetRxTMS_FaultLvl();
```

```
VCAN_TMS_PowerRqst = (INT16U)((((INT16U)IIGetRxTMS_ReqPower_Hbyte() << 8u)  
+ (INT16U)IIGetRxTMS_ReqPower_Lbyte()));
```

```
if (VCAN_TMS_PowerRqst == DEFAULT_UINT16)
```

```
{
```

```
    VCAN_TMS_PowerRqst = 0u;
```

```
}
```

```
VCAN_TMS_COMLost_Flag = FALSE;
```

```
(void)Stb_S_SetTimer(&TMS_timeout_timer);
```

```
}
```

```
else
```

```
{
```

```
    if (0u == IS_TMS_WAKE())
```

```
    {
```

```
        /* TMS 不唤醒：不判丢失 */
```

```
        VCAN_TMS_COMLost_Flag = FALSE;
```

```
        (void)Stb_S_SetTimer(&TMS_timeout_timer);
```

```
    }
```

```
    else if (FALSE == VCAN_TMS_COMLost_Flag)
```

```
    {
```

```
        if (Stb_S_Timeout(TMS_timeout_timer, TMS_TIMEOUT))
```

```
        {
```

```
            VCAN_TMS_COMLost_Flag = TRUE;
```

```
        }
```

```
    }
```

```
    else
```

```
    {
```

```
        (void)Stb_S_SetTimer(&TMS_timeout_timer);
```

```
    }
```

```
}
```

```
ApiPutTMSCommErr(VCAN_TMS_COMLost_Flag);
```

```
ApiPutThml_TMSPower(VCAN_TMS_PowerRqst);
ApiPutThml_TMSHVS(VCAN_TMS_RlyST);
ApiPutThml_TMSFaultLvl(VCAN_TMS_FaultLvl);
ApiPutThml_TMSInletTemp(VCAN_TMS_InputTemp);
ApiPutThml_TMSOutletTemp(VCAN_TMS_OutputTemp);
}
...
```

2.1.5 其他可选接口

- ``void ApiPutPowerOffCmd(INT8U cmd, VCU_CMD src);``
 - 整车控制低压供电：发送该命令约 5s 后整车切断低压。
 - ``VCU_CMD`` 为宏，表示来自 VCU 的低压下电指令源。

- ``void ApiPutHVEmergOffCmd(BOOLEAN EmergOffCmd);``（可选）
 - BMS 收到后直接断开继电器，不做电流判断；谨慎使用。

- ``void ApiPutThml_VehicleSpeed(INT16U VehicleSpeed);``（可选）
 - 单位 km/h，分辨率 1。

- ``void ApiPutVinByte(INT8U index, INT8U VinByte);``（可选）
 - 建议收齐 VIN 后，与 ``RteGetVinByte(i)`` 对比，不一致再更新，避免重复刷写。

- ``void ApiPutVCUDCChgActive(INT8U cmd);``（可选）
 - 直流充电 BRO 阶段：BMS 收到该指令后才闭合直流充电继电器。

- ``void ApiPutVCUDCChgAllow(INT8U cmd);``（可选）
 - 直流充电 BRO 阶段：收到允许后 BMS 才发送 ``0xAA`` 进入充电流程。
 - 充电中收到“不允许”：退出直流充电，BST 发送其他故障。

- ``void ApiPutInsulationCommErr(INT8U data);``（绝缘仪在整车CAN网络时）
- ``void ApiPutExtSystemIsoValue(INT16U isoval);``（绝缘仪在整车CAN网络时）
 - 单位 kΩ，分辨率 1。

2.2 发送报文制作

整车发送报文模块主要负责将电池系统的信息传给整车，针对大巴项目，发送报文信息还需要满足GB/T 32960内容要求。

以REPT_Protocol_VCAN_V3.0通讯协议为例，发送给整车的信息包含

2.2.1 常用外发接口列表

接口数据类型/原型	说明
-----------	----

INT16U RteGetTotalCurrentValue(void)	母线电流；单位A；分辨率0.1；偏移-3200A
INT16U RteGetTotalSoc(void)	系统SOC；单位%；分辨率0.1；偏移0
INT16U RteGetTotalSohValue(void)	系统SOH；单位%；分辨率0.1；偏移0
INT16U RteGetTotalRemained_Engy(void)	剩余能量；单位kWh；分辨率0.1；偏移0
INT16U RteGetSysInsulationValue(void)	系统绝缘值；单位kΩ；分辨率0.1；偏移0
INT16U RteGetPosInsulationValue(void)	系统正对地绝缘值；单位kΩ；分辨率0.1；偏移0
INT16U RteGetNegInsulationValue(void)	系统负对地绝缘值；单位kΩ；分辨率0.1；偏移0
INT16U RteGetTotalCellVoltSum(void)	系统累加和电压；单位V；分辨率0.1；偏移0
INT16U RteGetCellVoltSum(INT8U StringNo)	（多支路）单支路累加和电压；单位V；分辨率0.1；偏移0
INT16U RteGetCurrentValue(INT8U StringNo)	（多支路）单支路电流；单位A；分辨率0.1；偏移-3200A
BOOLEAN RteGetDCChgCnntST(void)	直流充电枪连接状态；0=未连接，1=已连接
BOOLEAN RteGetACChgCnntST(void)	交流充电枪连接状态；0=未连接，1=已连接
INT8U RteGetFcgStep(void)	直流充电步骤/状态机（用于计算充电状态：未充电/准备充电/正在充电/充电完成）
INT8U RteGetACCHGStep(void)	交流充电步骤/状态机（用于计算充电状态）
INT8U RteGetBMSDataStoredStatus(void)	数据存储完成标志；0=未完成，1=完成
BOOLEAN RteGetInitFailedFlag(void)	初始化失败标志（用于BMS高压状态机）
BOOLEAN RteGetMainHVReadyStatus(void)	主回路高压Ready状态（用于BMS高压状态机）
INT8U RteGetMainHVStatus(void)	主回路高压状态；0=断开，1=预充，2=上高压中，3=闭合，4=预充失败，5=上高压错误
INT8U RteGetMainNegCntST(void)	主负继电器故障/状态（用于组装继电器状态、故障位）；典型：0=正常，1/2=故障语义依项目定义
INT8U RteGetMainPosCntST(void)	主正继电器故障/状态（用于组装继电器状态、故障位）
INT8U RteGetDCChgPos1CntST(void)	直流充电正继电器1故障/状态
INT8U RteGetDCChgNeg1CntST(void)	直流充电负继电器1故障/状态
INT8U RteGetDCChgPos2CntST(void)	直流充电正继电器2故障/状态
INT8U RteGetDCChgNeg2CntST(void)	直流充电负继电器2故障/状态
INT8U RteGetHeatPosCntST(void)	加热正继电器故障/状态
INT8U RteGetHeatNegCntST(void)	加热负继电器故障/状态
INT8U RteGetTMSCntST(void)	水冷机组继电器故障/状态
INT8U RteGetVehicleFaultLevel(void)	BMS最高故障等级；0=无故障，1=轻微，2=一般，3=严重
INT32U RteGetTroubleCode(void)	最严重故障码（具体编码规则按项目/协议定义）
INT32U RteGetVehCyclicTroubleCode(void)	循环故障码；每调用一次更新一个故障码
INT8U RteGetDCChgStopCode(void)	充电停止原因（总故障码）
BOOLEAN RteGetDCChgCommErr(void)	直流充电通讯超时故障标志
INT8U RteGetChgrStopReason(INT8U index)	充电机停止充电故障（对应CST），index按协议定义
INT8U RteGetDCChgStopReason(INT8U index)	BMS停止充电故障（对应BST），index按协议定义
INT8U RteGetCellTemperHighErr(void)	单体温度过高故障等级；0~3
INT8U RteGetCellTemperLowErr(void)	单体温度过低故障等级；0~3
INT8U RteGetCellTemperDiffErr(void)	温差过大故障等级；0~3
INT8U RteGetCellVoltMaxErr(void)	单体电压过高故障等级；0~3
INT8U RteGetCellVoltMinErr(void)	单体电压过低故障等级；0~3
INT8U RteGetCellVoltDiffErr(void)	单体压差过大故障等级；0~3
INT8U RteGetPackVoltHighErr(void)	电池组电压过高故障等级；0~3
INT8U RteGetPackVoltLowErr(void)	电池组电压过低故障等级；0~3
INT8U RteGetDchPeakCurrHighErr(void)	放电电流过流故障等级；0~3

INT8U RteGetFbChgPeakCurrHighErr(void)	回充电流过流故障等级；0~3
INT8U RteGetSocLowErr(void)	SOC过低故障等级；0~3
INT8U RteGetInsulationErr(void)	绝缘过低故障等级；0~3
INT8U RteGetPowerSupplyLowErr(void)	供电电压过低故障等级；0~3
INT8U RteGetPowerSupplyHighErr(void)	供电电压过高故障等级；0~3
INT8U RteGetChgSusCurrHighErr(void)	充电电流过流故障等级；0~3
BOOLEAN RteGetBranchOpenErr(void)	支路开路故障标志（多支路）；0=无故障，1=有故障
INT8U RteGetPackVoltMismatchErr(void)	MSD断开/电压不匹配相关状态（示例中：==3u判为MSD断开）
BOOLEAN RteGetHvilAErr(void)	高压互锁故障标志
BOOLEAN RteGetFireErr(void)	火灾报警标志
BOOLEAN RteGetSlaverCommErr(void)	从板通讯丢失故障标志
BOOLEAN RteGetCurrSensorCommErr(void)	电流传感器采样/通讯故障标志
BOOLEAN RteGetBalanceErr(void)	均衡故障标志
BOOLEAN RteGetHeatingSysErr(void)	加热膜回路故障标志
BOOLEAN RteGetVcuCommErr(void)	与VCU通讯丢失故障标志
BOOLEAN RteGetDCDCErr(void)	DCDC故障标志（电池系统自带DCDC时）
BOOLEAN RteGetDCSocketANTCErr(void)	直流充电插座A NTC故障标志
BOOLEAN RteGetDCSocketBNTCErr(void)	直流充电插座B NTC故障标志
BOOLEAN RteGetDCSocketOverHigh2(void)	直流充电插座NTC过温二级标志
BOOLEAN RteGetDCSocketOverHigh1(void)	直流充电插座NTC过温一级标志
INT16U RteGetTotalDchPeakCurrLimit(void)	SOP：最大允许瞬时代放电电流；单位A；分辨率0.1；（需配置SOP_DEFAULT_INDEX）
INT16U RteGetTotalDchCurrLimit(void)	SOP：最大允许持续放电电流；单位A；分辨率0.1
INT16U RteGetTotalChgPeakCurrLimit(void)	SOP：最大允许瞬时充电电流；单位A；分辨率0.1；（需配置SOP_DEFAULT_INDEX）
INT16U RteGetTotalChgCurrLimit(void)	SOP：最大允许持续充电电流；单位A；分辨率0.1
INT16U RteGetTotalDchPeakPowerLimit(void)	SOP：最大允许瞬时代放电功率；单位kW；分辨率0.1；（需配置SOP_DEFAULT_INDEX）
INT16U RteGetTotalDchPowerLimit(void)	SOP：最大允许持续放电功率；单位kW；分辨率0.1
INT16U RteGetTotalChgPeakPowerLimit(void)	SOP：最大允许瞬时充电功率；单位kW；分辨率0.1；（需配置SOP_DEFAULT_INDEX）
INT16U RteGetTotalChgPowerLimit(void)	SOP：最大允许持续充电功率；单位kW；分辨率0.1
INT16U RteGetTotalDchPeakCurrLimit10S(void)	SOP：最大允许10s瞬时代放电电流；单位A；分辨率0.1
INT16U RteGetTotalChgPeakCurrLimit10S(void)	SOP：最大允许10s瞬时充电电流；单位A；分辨率0.1
INT16U RteGetTotalDchPeakCurrLimit30S(void)	SOP：最大允许30s瞬时代放电电流；单位A；分辨率0.1
INT16U RteGetTotalChgPeakCurrLimit30S(void)	SOP：最大允许30s瞬时充电电流；单位A；分辨率0.1
INT16U RteGetTotalDchPeakCurrLimit60S(void)	SOP：最大允许60s瞬时代放电电流；单位A；分辨率0.1
INT16U RteGetTotalChgPeakCurrLimit60S(void)	SOP：最大允许60s瞬时充电电流；单位A；分辨率0.1
INT16U RteGetTotalCellVoltMaxMax(void)	最高单体电压
INT16U RteGetTotalCellVoltMinMin(void)	最低单体电压
INT16U RteGetTotalCellVoltMean(void)	单体平均电压
INT16U RteGetTotalCellVoltDiff(void)	单体电压差
INT16U RteGetTotalCellTempMaxMax(void)	最高单体温度
INT16U RteGetTotalCellTempMinMin(void)	最低单体温度
INT8U RteGetTotalCellTempMean(void)	单体平均温度
INT16U RteGetTotalCellTempDiff(void)	单体温差

INT8U RteGetTotalCellVoltMaxModule(INT8U index)	最高单体电压所在模组号 (index通常固定0u取最值)
INT8U RteGetTotalCellVoltMaxNum(INT8U index)	最高单体电压所在模组内电芯号
INT8U RteGetTotalCellVoltMinModule(INT8U index)	最低单体电压所在模组号
INT8U RteGetTotalCellVoltMinNum(INT8U index)	最低单体电压所在模组内电芯号
INT8U RteGetTotalCellTempMaxModule(INT8U index)	最高单体温度所在模组号
INT8U RteGetTotalCellTempMaxNum(INT8U index)	最高单体温度所在模组内电芯号
INT8U RteGetTotalCellTempMinModule(INT8U index)	最低单体温度所在模组号
INT8U RteGetTotalCellTempMinNum(INT8U index)	最低单体温度所在模组内电芯号
INT16U RteGetTotalCellVoltMaxPos(INT8U index)	最高单体电压序号 (所有电芯的全局序号)
INT16U RteGetTotalCellVoltMinPos(INT8U index)	最低单体电压序号 (所有电芯的全局序号)
INT16U RteGetTotalCellTempMaxPos(INT8U index)	最高单体温度序号 (所有电芯的全局序号)
INT16U RteGetTotalCellTempMinPos(INT8U index)	最低单体温度序号 (所有电芯的全局序号)
INT16U RteGetCellVoltage(INT8U StringNo, INT16U index)	详细电压: 单体电压读取接口 (用于单帧循环发送/动态ID发送)
INT8U RteGetCellTemp(INT8U StringNo, INT16U index)	详细温度: 单体温度读取接口 (用于单帧循环发送/动态ID发送)
INT8U RteGetSysManufactureYear(void)	产品信息: 生产日期-年 (0~99, 按项目约定基准年)
INT8U RteGetSysManufactureMonth(void)	产品信息: 生产日期-月 (1~12)
INT8U RteGetSysManufactureDay(void)	产品信息: 生产日期-日 (1~31)
INT8U RteGetPacksSn(INT8U index)	产品信息: 系统编码SN字节; index < SN_LENGTH
INT32U RteGetTotalAccuChargeEnergy(void)	累计充电能量; 单位kWh; 分辨率0.1
INT32U RteGetTotalAccuDischargeEnergy(void)	累计放电能量; 单位kWh; 分辨率0.1
INT32U RteGetTotalAccuFeedChargeEnergy(void)	累计回充能量; 单位kWh; 分辨率0.1
INT32U RteGetTotalSingleChargeEnergy(void)	单次充电能量; 单位kWh; 分辨率0.1
INT32U RteGetTotalSingleDischargeEnergy(void)	单次放电能量; 单位kWh; 分辨率0.1
INT32U RteGetTotalSingleFeedChargeEnergy(void)	单次回充能量; 单位kWh; 分辨率0.1
INT8U RteGetVinByte(INT8U index)	发送BMS存储的VIN码字节 (与接收ApiPutVinByte对应)
INT16U Rte_Get_ACCHGVoltRequest(void)	交流充电OBC报文: 请求电压; 单位V; 分辨率0.1; 偏移0
INT16U Rte_Get_ACCHGCurRequest(void)	交流充电OBC报文: 请求电流; 单位A; 分辨率0.1; 偏移0
INT8U Rte_Get_ACCHGCMD(void)	交流充电OBC报文: 充电命令; 0=停止充电, 1=开启充电
INT8U RteGetThml_TMSMode(void)	热管理: 工作模式; 0=关机, 1=冷却, 2=加热, 3=自循环
INT8U RteGetThml_TMSHVCmd(void)	热管理: TMS继电器闭合指令; 0=断开, 1=闭合
INT8U RteGetThml_TMSLqdTempSet(void)	热管理: 目标温度设定; 单位°C; 分辨率1; 偏移-40°C

2.2.2 充电状态发送

参考代码示例:

```

``c
/*
 * chgstatus:
 * 0 = 未充电
 * 1 = 准备充电
 * 2 = 正在充电
 * 3 = 充电完成
 */

if ((DCCHG_STEP_HANDSHAKE <= RteGetFcgStep())

```

```
|| (ACCHG_STEP_PREPARING <= RteGetACCHGStep()))
{
    if ((DCCHG_STEP_CHARGING == RteGetFcgStep())
        || (ACCHG_STEP_CHARGING == RteGetACCHGStep()))
    {
        chgstatus = 2u;
    }
    else if ((DCCHG_STEP_STOPCHARGING <= RteGetFcgStep())
             || (ACCHG_STEP_STOPCHARGING <= RteGetACCHGStep()))
    {
        chgstatus = 3u;
    }
    else
    {
        chgstatus = 1u;
    }
}
else
{
    chgstatus = 0u;
}
...

```

2.2.3 高压状态发送

参考代码示例:

```
```c
/*
 * BmsHVStatus:
 * 0 初始化
 * 1 Standby (准备上高压)
 * 2 上高压完成
 * 3 上高压失败
 * 4 高压断开
 * 5 初始化失败
 */
switch (BmsHVStatus)
{

```

```
case 0u: /* Initializing */
{
 if (RteGetInitFailedFlag())
 {
 BmsHVStatus = 5u;
 }
 else if (RteGetMainHVReadyStatus())
 {
 BmsHVStatus = 1u;
 }
}
break;

case 1u: /* Standby, ready to HV on */
{
 if ((HV_ERR == RteGetMainHVStatus())
 || (HV_PRECHGFAILURE == RteGetMainHVStatus()))
 {
 BmsHVStatus = 3u;
 }
 else if (HV_ON == RteGetMainHVStatus())
 {
 BmsHVStatus = 2u;
 }
}
break;

case 2u: /* HV ON, main contactors closed */
{
 if (HV_OFF == RteGetMainHVStatus())
 {
 BmsHVStatus = 4u;
 }
}
break;

case 3u: /* Failed to HV on */
{
 /* Wait to power-off */
```

```
}

break;

case 4u:
{
 if (RteGetMainHVReadyStatus())
 {
 BmsHVStatus = 1u;
 }
}

break;

case 5u: /* Initialize failed */
{
 if (RteGetMainHVReadyStatus())
 {
 BmsHVStatus = 1u;
 }
}

break;

default:
 BmsHVStatus = 0u;
 break;
}
...

```

## 2.2.4 详细电压发送（单帧循环）

参考代码示例:

每帧发送 4 个电芯电压。

```
```c  
  
#define VCAN_UCELLCOUNT_PERFRAM (4u) /* Cell volt num sent by every frame */  
  
static void VCanBMS_CyclicCellVolt_Sendout(void)
```

```
{  
  
    static INT8U FrameNo = DEFAULT_UINT8;  
  
    static INT8U VoltStringPos;  
  
  
    INT16U VoltCellPos;  
  
  
    if (IIGetUcell_Fram_NoConfirmation())  
    {  
        IIClrUcell_Fram_NoConfirmation();  
  
        /* 计算下一帧序号 */  
        if (((INT16U)(FrameNo + 1u) * VCAN_UCELLNUM_PERFRAM)  
            < (INT16U)PRO_PACK_PARELL_NUM * PRO_CELL_VOLT_SERIAL)  
        {  
            FrameNo++;  
        }  
        else  
        {  
            FrameNo = 0u;  
            VoltStringPos = 0u;  
        }  
  
        IIPutTxUcell_Fram_No(FrameNo);  
  
        /* 起始 cell 位置 */  
        VoltCellPos = (INT16U)VCAN_UCELLNUM_PERFRAM * FrameNo;  
  
        VoltStringPos = (INT8U)(VoltCellPos / PRO_CELL_VOLT_SERIAL);  
        VoltCellPos = (INT16U)(VoltCellPos % PRO_CELL_VOLT_SERIAL);  
  
        /* 依次发送 4 个电芯电压；超过范围则填无效值 */  
        IIPutTxUcell1(RteGetCellVoltage(VoltStringPos, VoltCellPos));  
  
        VoltCellPos++;  
        if (((INT16U)VoltStringPos * PRO_CELL_VOLT_SERIAL + VoltCellPos)  
            >= (INT16U)PRO_PACK_PARELL_NUM * PRO_CELL_VOLT_SERIAL)  
        {  
            IIPutTxUcell2(DEFAULT_UINT16);  
        }  
    }  
}
```

```

else
{
    IIPutTxUcell2(RteGetCellVoltage(VoltStringPos, VoltCellPos));
}

VoltCellPos++;
if (((INT16U)VoltStringPos * PRO_CELL_VOLT_SERIAL + VoltCellPos)
    >= (INT16U)PRO_PACK_PARELL_NUM * PRO_CELL_VOLT_SERIAL)
{
    IIPutTxUcell3(DEFAULT_UINT16);
}
else
{
    IIPutTxUcell3(RteGetCellVoltage(VoltStringPos, VoltCellPos));
}

VoltCellPos++;
if (((INT16U)VoltStringPos * PRO_CELL_VOLT_SERIAL + VoltCellPos)
    >= (INT16U)PRO_PACK_PARELL_NUM * PRO_CELL_VOLT_SERIAL)
{
    IIPutTxUcell4(DEFAULT_UINT16);
}
else
{
    IIPutTxUcell4(RteGetCellVoltage(VoltStringPos, VoltCellPos));
}
}
}
...

---
```

2.2.5 详细电压发送（动态 ID）

参考代码示例:

每帧发送 4 个电芯电压；通过动态 ID 携带帧号。

```
```c
```

```
#include "cancntrl.h"

#define VCAN_UCELL_DynHdle CanTxTxDynamicMsg0
#define VCAN_UCELL_ID (0x1820F3F4u)
#define VCAN_UCELL_IDINCPOS (16u)
#define VCAN_UCELLNUM_PERMSG (4u)

static CAN_MSG CellVoltMsgPakage;
static INT8U guarrVCAN_UCelldata[8u] = {0u};

static BOOLEAN VCanDynamicTransmit_CellVolt(const CAN_MSG *pMsg)
{
 INT8U *dest;

 if ((NULL == pMsg) || (NULL == pMsg->bDb))
 {
 return FALSE;
 }

 /* 动态对象设置：ExtId、DLC、Data */
 CanDynTxObjSetExtId(VCAN_UCELL_DynHdle,
 (INT16U)((pMsg->qblId.dw & 0xFFFF0000u) >> 16),
 (INT16U)(pMsg->qblId.dw & 0x0000FFFFu));
 CanDynTxObjSetDlc(VCAN_UCELL_DynHdle, (INT8U)pMsg->Dlc);

 dest = (INT8U*)CanGetTxDataPtr(VCAN_UCELL_DynHdle);
 (void)memcpy((INT8U*)dest, (INT8U*)pMsg->bDb, pMsg->Dlc);

 return CanTransmit(VCAN_UCELL_DynHdle);
}

static void VCanBMS_DynmCellVolt_Sendout(void)
{
 static INT8U UCellMsgNo = DEFAULT_UINT8;

 INT16U basePos;
 INT16U totalCellNum;
 INT8U i;
```

```
totalCellNum = (INT16U)PRO_PACK_PARELL_NUM * PRO_CELL_VOLT_SERIAL;
```

```
/* 只有上一帧发送成功才推进帧号 */
```

```
if (VCanDynamicTransmit_CellVolt(&CellVoltMsgPakage))
```

```
{
```

```
 if (((INT16U)(UCellMsgNo + 1u) * VCAN_UCELLNUM_PERMSG) < totalCellNum)
```

```
 {
```

```
 UCellMsgNo++;
```

```
 }
```

```
 else
```

```
 {
```

```
 UCellMsgNo = 0u;
```

```
 }
```

```
 basePos = (INT16U)VCAN_UCELLNUM_PERMSG * UCellMsgNo;
```

```
 for (i = 0u; i < VCAN_UCELLNUM_PERMSG; i++)
```

```
 {
```

```
 INT16U absPos = (INT16U)(basePos + i);
```

```
 if (absPos >= totalCellNum)
```

```
 {
```

```
 /* 超出总电芯数：填无效 */
```

```
 guarrVCAN_UCelldata[2u * i] = DEFAULT_UINT8;
```

```
 guarrVCAN_UCelldata[2u * i + 1] = DEFAULT_UINT8;
```

```
 }
```

```
 else
```

```
 {
```

```
 INT8U stringNo = (INT8U)(absPos / PRO_CELL_VOLT_SERIAL);
```

```
 INT16U cellPos = (INT16U)(absPos % PRO_CELL_VOLT_SERIAL);
```

```
 INT16U val = RteGetCellVoltage(stringNo, cellPos);
```

```
 guarrVCAN_UCelldata[2u * i] = (INT8U)(val & 0x00FFu);
```

```
 guarrVCAN_UCelldata[2u * i + 1] = (INT8U)((val & 0xFF00u) >> 8u);
```

```
 }
```

```
 }
```

```
 CellVoltMsgPakage.qbld.dw = (DWORD)(VCAN_UCELL_ID + ((INT32U)UCellMsgNo <<
VCAN_UCELL_IDINCPOS));
```

```

CellVoltMsgPakage.bDb = (BYTE *)&guarrVCAN_UCelldata[0];
CellVoltMsgPakage.Dlc = 8u;
CellVoltMsgPakage.Rtr = 0u;
CellVoltMsgPakage.Ext = 1u;
CellVoltMsgPakage.TxOk = 0u;
}
}
```


---


```

2.2.6 详细温度发送（单帧循环）

参考代码示例:

```

```c
#define VCAN_TCELLNUM_PERFRAM (7u) /* temp sensor num sent by every frame */

static void VCanBMS_CyclicCellTemp_Sendout(void)
{
 static INT8U FrameNo = DEFAULT_UINT8;
 static INT8U TempStringPos;

 INT16U TempSnsrPos;

 if (ILGetTcell_Fram_NoConfirmation())
 {
 ILClrTcell_Fram_NoConfirmation();

 if (((INT16U)(FrameNo + 1u) * VCAN_TCELLNUM_PERFRAM)
 < (INT16U)PRO_PACK_PARELL_NUM * PRO_CELL_TEMP_SERIAL)
 {
 FrameNo++;
 }
 else
 {
 FrameNo = 0u;
 TempStringPos = 0u;
 }
 }
}
```

```

```
IlPutTxTcell_Fram_No(FrameNo);
```

```
TempSnsrPos = (INT16U)VCAN_TCELLNUM_PERFRAM * FrameNo;
```

```
TempStringPos = (INT8U)(TempSnsrPos / PRO_CELL_TEMP_SERIAL);
```

```
TempSnsrPos = (INT16U)(TempSnsrPos % PRO_CELL_TEMP_SERIAL);
```

```
/* 示例：依次发送 7 个温度；超出范围填无效 */
```

```
IlPutTxTemp1(RteGetCellTemp(TempStringPos, TempSnsrPos));
```

```
/* 其余 Temp2~Temp7 的边界判断与 Temp1 同理，可按电压示例方式写 */
```

```
}
```

```
}
```

```
...
```

2.2.7 详细温度发送（动态 ID）

参考代码示例:

```
```c
```

```
#include "cancntrl.h"
```

```
#define VCAN_TCELL_DynHdle CanTxTxDynamicMsg1
```

```
#define VCAN_TCELL_ID (0x18A0F3F4u)
```

```
#define VCAN_TCELL_IDINCPOS (16u)
```

```
#define VCAN_TCELLNUM_PERMSG (8u)
```

```
static CAN_MSG CellTempMsgPackage;
```

```
static INT8U guarrVCAN_TCelldata[8u] = {0u};
```

```
static BOOLEAN VCanDynamicTransmit_CellTemp(const CAN_MSG *pMsg)
```

```
{
```

```
 INT8U *dest;
```

```
 if ((NULL == pMsg) || (NULL == pMsg->bDb))
```

```
 {
```

```
 return FALSE;
```

```
 }
```

```
CanDynTxObjSetExtId(VCAN_TCELL_DynHdle,
 (INT16U)((pMsg->qblId.dw & 0xFFFF0000u) >> 16),
 (INT16U)(pMsg->qblId.dw & 0x0000FFFFu));
CanDynTxObjSetDlc(VCAN_TCELL_DynHdle, (INT8U)pMsg->Dlc);

dest = (INT8U*)CanGetTxDataPtr(VCAN_TCELL_DynHdle);
(void)memcpy((INT8U*)dest, (INT8U*)pMsg->bDb, pMsg->Dlc);

return CanTransmit(VCAN_TCELL_DynHdle);
}

static void VCanBMS_DynmCellTemp_Sendout(void)
{
 static INT8U TCellMsgNo = DEFAULT_UINT8;

 INT16U basePos;
 INT16U totalTempNum;
 INT8U i;

 totalTempNum = (INT16U)PRO_PACK_PARELL_NUM * PRO_CELL_TEMP_SERIAL;

 if (VCanDynamicTransmit_CellTemp(&CellTempMsgPakage))
 {
 if (((INT16U)(TCellMsgNo + 1u) * VCAN_TCELLNUM_PERMSG) < totalTempNum)
 {
 TCellMsgNo++;
 }
 else
 {
 TCellMsgNo = 0u;
 }

 basePos = (INT16U)VCAN_TCELLNUM_PERMSG * TCellMsgNo;

 for (i = 0u; i < VCAN_TCELLNUM_PERMSG; i++)
 {
 INT16U absPos = (INT16U)(basePos + i);

 if (absPos >= totalTempNum)
```

```
{
 guarrVCAN_TCelldata[i] = DEFAULT_UINT8;
}
else
{
 INT8U stringNo = (INT8U)(absPos / PRO_CELL_TEMP_SERIAL);
 INT16U tempPos = (INT16U)(absPos % PRO_CELL_TEMP_SERIAL);
 guarrVCAN_TCelldata[i] = RteGetCellTemp(stringNo, tempPos);
}
}

CellTempMsgPackage.qbld.dw = (DWORD)(VCAN_TCELL_ID + ((INT32U)TCellMsgNo <<
VCAN_TCELL_IDINCPOS));

CellTempMsgPackage.bDb = (BYTE *)&guarrVCAN_TCelldata[0];
CellTempMsgPackage.Dlc = 8u;
CellTempMsgPackage.Rtr = 0u;
CellTempMsgPackage.Ext = 1u;
CellTempMsgPackage.TxOk = 0u;
}
}
...
```

## 3 整车CAN测试

### 3.1 代码静态测试

使用QAC测试，提供测试报告

### 3.2 代码动态测试

使用Tessy测试，提供测试报告

### 3.3 HIL测试

台架测试，测试内容包含：报文数量、报文周期，报文信号（特别关注状态信号），提供HIL测试报告。

### 3.4 实车调试

按照整车测试报告，测试整车CAN相关模块，提供实车调试报告。